PHILOSOPHICAL TRANSACTIONS A

royalsocietypublishing.org/journal/rsta

Research

Article submitted to journal

Subject Areas:
xxxxx, xxxxx, xxxx

Keywords:
symbolic regression, genetic programming, Zobrist hash, tree hashing, diversity, fitness caching

Author for correspondence:
Bogdan Burlacu
e-mail: bogdan.burlacu@fh-ooe.at

Zobrist Hash-based Duplicate Detection in Symbolic Regression

Bogdan Burlacu

University of Applied Sciences Upper Austria
Heuristic and Evolutionary Algorithms Laboratory
Softwarepark 11, 4232, Hagenberg, Austria

Symbolic regression encompasses a family of search algorithms that aim to discover the best fitting function for a set of data without requiring an *a priori* specification of the model structure. The most successful and commonly used technique for symbolic regression is Genetic Programming (GP), an evolutionary search method that evolves a population of mathematical expressions through the mechanism of natural selection. In this work we analyze the efficiency of the evolutionary search in GP and show that many points in the search space are re-visited and re-evaluated multiple times by the algorithm, leading to wasted computational effort. We address this issue by introducing a caching mechanism based on the Zobrist hash, a type of hashing frequently used in abstract board games for the efficient construction and subsequent update of transposition tables. We implement our caching approach using the open-source framework Operon and demonstrate its performance on a selection of real-world regression problems, where we observe up to 34% speedups without any detrimental effects on search quality. The hashing approach represents a straightforward way to improve runtime performance while also offering some interesting possibilities for adjusting search strategy based on cached information.

THE ROYAL SOCIETY PUBLISHING

1. Introduction

Symbolic Regression (SR) has been rapidly emerging as a key approach in machine learning in recent years due to the widespread acceptance of interpretability as an unequivocally crucial element of trust in AI [1]. Compared to "black-box" models such as deep neural networks or support vector machines with nonlinear kernels, whose decision-making processes are opaque and not easily understood by humans, models produced by SR are given in the form of mathematical expressions that are, to a large extent, fathomable, simple and insightful. Its "white-box" characteristics make SR an invaluable tool for scientific discovery in many application domains, such as astrophysics [2,3], physical law discovery [4–6], materials science [7–9], particle kinematics [10,11], robotics [12], controller design [13], or clinical decision support [14]. SR distinguishes itself from other regression methods that are also capable of producing symbolic models (e.g. polynomial models up to a given degree) by its ability to learn from data without requiring an *a priori* specification of the model structure. In SR, both model structure and parameters are simultaneously developed, allowing for much greater expressive power and flexibility.

Despite a recent emergence of many competing approaches, based on deep learning [15–18], neural networks [19], grammar enumeration [20], mixed-integer nonlinear programming [21], path-wise regularized learning [22], or exhaustive search [3,23], Genetic Programming [24] (GP), a biologically-inspired metaheuristic operating after the model of natural selection, remains the most successful and overall best performing approach for SR [25,26]. However, while undeniably powerful, GP-based SR still suffers from the intrinsic limitations of evolutionary methods, such as high computational cost, search inefficiency [27], diversity loss and risk of premature convergence [28,29]. These limitations are further compounded by the non-deterministic nature of the search which makes it necessary to perform multiple algorithmic runs to get reliable estimates.

In this work we introduce an approach for mitigating these limitations by caching duplicate expressions encountered during the search, precluding the need for solution re-evaluation and saving execution time. Our method is based on the Zobrist hash, commonly used to detect transpositions in board games like Chess and Go. In our case, a transposition is defined as a sequence of recombination operations (crossover or mutation) that leads to an already-seen expression. In what follows, we show that transpositions occur frequently in SR.

The paper is structured as follows. Section 2 offers a more detailed discussion of GP and argues why duplicates do occur during the search. Section 3 explains the proposed duplicate detection mechanism and its implementation. Section 4 provides empirical evidence in favor of fitness caching as a way to improve runtime, also noting a change in search dynamics with caching. Finally, Section 5 discusses the implications of this work and future research directions.

2. Symbolic Regression with Genetic Programming

Genetic Programming [24] is a metaheuristic optimization method that performs a *global search* in solution space by orchestrating the evolution of a *population* of computer programs using the basic mechanisms of *selection* and *recombination*. Each program, called an *individual*, in keeping with the biological metaphor, contains instructions for solving the problem at hand and its *fitness* measures its ability to achieve the problem objectives[1]. Starting with a random initial population, in each iteration of the algorithm, the fittest individuals are selected for reproduction and subsequently swap parts of their structure to produce potentially fitter offspring. The process continues for a fixed number of iterations or until convergence, according to the workflow described in Figure 1.

In GP-based SR, the tree representation encodes mathematical expressions, as illustrated in Figure 2. SR performs a search of the space of mathematical expressions by gradually assembling tree structures from a pre-configured set of functions and terminals. Given a set of binary (e.g. $+, -, \times, \div, \mathrm{pow}, \mathrm{aq}, \dots$), unary (e.g. $\sin, \cos, \exp, \log, \sqrt{\cdot}, \dots$) and nullary (i.e.

[1]The actual representation and fitness measure are problem-dependent.

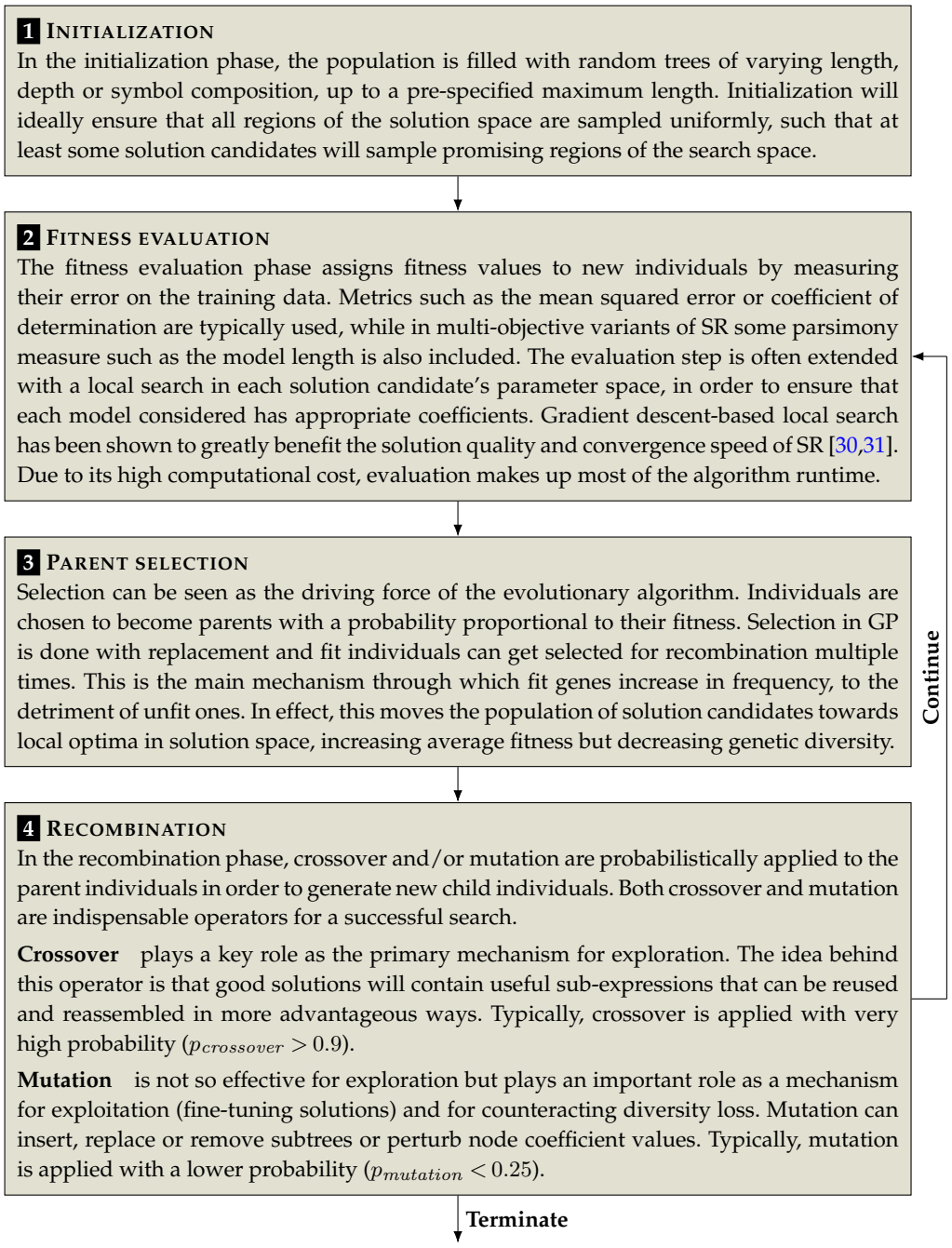


Figure 1: Evolutionary workflow of Genetic Programming

variables and parameters) operators, the search space is defined as all valid compositions of operators (subject to some size constraints), making it a discrete combinatorial search space whose dimension scales exponentially with the number of operators.

The successful exploration of the search space depends on the existence of a sufficiently diverse set of individuals in the population, from which adaptive change can emerge as the product of recombination. At the same time, successful *exploitation*, understood as the local improvement of solutions in the vicinity of optima points, requires a focusing of the search effort and depends

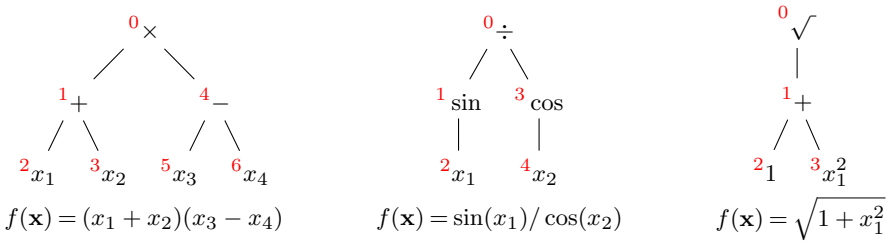


Figure 2: Example tree representations with preorder traversal indices shown in red.

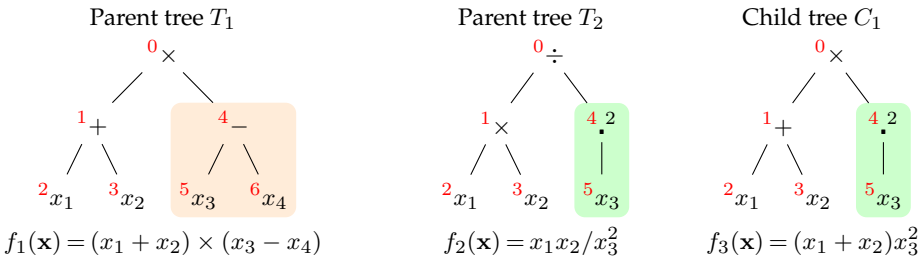


Figure 3: Crossover between two parent tree-encoded expressions. A new symbolic expression $f_3(\mathbf{x}) = (x_1 + x_2)x_3^2$ is obtained by exchanging the parts $(x_3 - x_4)$ and x_3^2 between the parents.

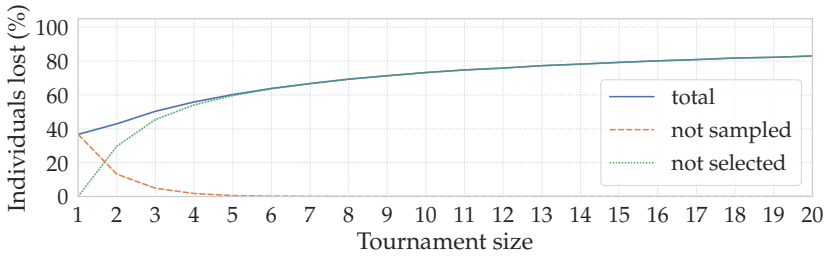


Figure 4: Percentage of not selected individuals as a function of tournament size

on the selection mechanism's ability to concentrate the population on promising regions of the search space. A balance between these two antagonistic aspects of the search is difficult to achieve since it requires a deep understanding of evolutionary dynamics and is, in general, problem-dependent [32]. A good trade-off depends on appropriate settings for parameters such as the population size, crossover and mutation probabilities or selection pressure. The latter plays a particularly important role, since it controls the survival and reproduction rates of fit individuals.

Selection in a finite population causes diversity loss due to duplication of fit individuals [33]. In tournament selection, the most popular selection mechanism in SR, this effect was shown to be significant even at small tournament sizes [34]. A simulation of tournament selection in Figure 4 shows that at least 40% of individuals are lost in every generation. Diversity has an impact on search convergence, as the outcome of recombination will depend on the amount of genetic material shared by the parents. The more similar the parents, the more likely a significant amount of duplicate material will be inherited by the child.

From a practical perspective, given the unavoidable loss of diversity over the course of evolution, it is reasonable to assume that some solutions produced by recombination will not be unique, and their initial evaluations could be reused, with significant runtime benefits.

Furthermore, detecting when duplicates *may* occur or detecting the *frequency* of duplicate individuals in the population may also lead to better strategies for diversity maintenance.

Previous attempts to improve runtime efficiency focused on alternative representations, for example storing the population as a graph [35–37] or subtree caching [38]. The caching approach is of particular interest here as it does not require changes to the fundamental structure of the algorithm. However, implementing an efficient cache requires an accurate, low-overhead hashing function, with certain desirable properties such as hash involution. This paper introduces a hashing approach based on Zobrist keys that allows the caching of expression trees with minimal overhead, whose inner workings are described in the following section.

3. Duplicate Detection in Symbolic Regression

Duplicate detection in SR populations is challenging because of the need to compute distances between unordered labeled trees, a problem known to be NP-complete [39]. However, if one relaxes the problem to approximate matching, then hash-based tree isomorphism offers similar results with a marginal error bounded by the collision rate[2]. Hash functions based on modular arithmetic [38] or Merkle trees [40] have been previously used in SR to improve runtime and detect structural and semantic duplicates, but due to their logarithmic time complexity, their overhead remains significant.

Zobrist hashing, named after its inventor Albert Zobrist, is a hashing technique primarily used in computer game algorithms, particularly for board games such as Chess and Go. At its core, Zobrist hashing involves assigning a unique random number to each possible piece on each possible square of the game board. For instance, in chess, each combination of piece type and board position is mapped to a distinct random number. The hash value of a particular board state is then computed by performing a bitwise XOR (exclusive OR) operation on the numbers corresponding to the pieces present on the board.

The Zobrist hash has excellent runtime properties as XOR operations are generally very well-optimized on modern CPUs due to their simplicity and suitability for parallel execution. But the main advantage of Zobrist hashing becomes apparent when considering the mathematical properties of the XOR function, namely associativity, commutativity and involution.

$$a \oplus b = b \oplus a \qquad \text{(associativity)}$$

$$a \oplus (b \oplus c) = (a \oplus b) \oplus c \qquad \text{(commutativity)}$$

$$a \oplus b \oplus b = a \qquad \text{(involution)}$$

Given an abstract board game whose state can be reached through different move orders, the XOR function enables the incremental update of the game state by only considering changes to the board position after a move is made. In other words, previously visited states can be identified regardless of move order. This helps tree search algorithms such as *Minimax* avoid already-visited nodes in the search tree, by maintaining a data structure called a *transposition table* – a database that stores results of previously performed searches – greatly improving search efficiency.

The same approach is applicable to SR, considering the tree node type and its preorder index as the tabular coordinates for retrieving the associated Zobrist key, as illustrated in Figure 5. When two parent trees are crossed-over, the child hash will not be recomputed but instead updated with the hashes of the swapped subtrees[3]. For example, the hash of the child individual from Figure 3 is obtained by:

$$\mathcal{H}(C_1) = \mathcal{H}(T_1) \oplus \underbrace{h_{-,4} \oplus h_{x_3,5} \oplus h_{x_4,6}}_{\text{XOR "out" } (x_3 - x_4)} \oplus \underbrace{h_{\bullet^2,4} \oplus h_{x_3,5}}_{\text{XOR "in" } x_3^2}$$

[2]Perfect hashes are not considered due to their high computational cost.
[3]The incremental update will be faster when the cumulated length of the swapped subtrees is smaller than the length of T_1.

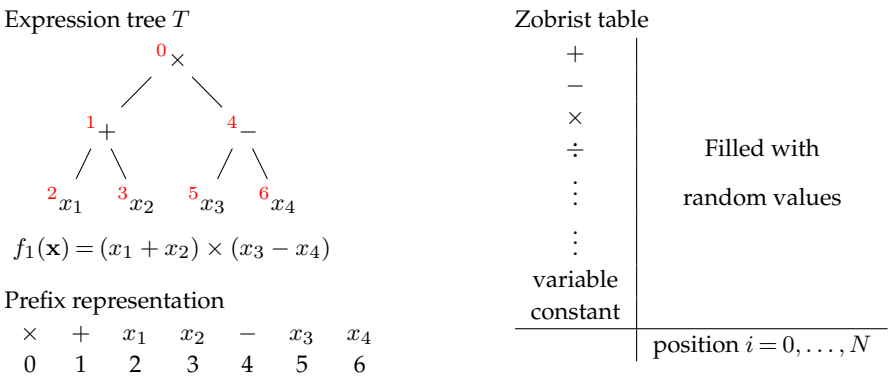


Figure 5: $\mathcal{H}(T) = h_{\times,0} \oplus h_{+,1} \oplus h_{x_1,2} \oplus h_{x_2,3} \oplus h_{-,4} \oplus h_{x_3,5} \oplus h_{x_4,6}$

Implementation

We base our implementation on the open-source library Operon [41] that contains a high-performance, concurrent implementation of SR in the C++ programming language. The library makes use of a fine-grained concurrency model where recombination and subsequent fitness evaluation take place in parallel, such that new individuals are generated independently in separate logical threads. In order to not diminish the concurrency and scalability of the library, interposing a fitness cache between the recombination and evaluation steps requires that the cache object is able to operate in a synchronized manner – that is, the underlying hash map object must support concurrent read/write operations. In order to satisfy there requirements we employ a parallel hash map library [42] with the following characteristics:

- *closed hashing*: the values are stored directly in a memory array (avoiding memory indirection)
- *data-parallel lookup*: using vector instructions, the hash table is able to look up items by checking 16 slots in parallel
- *segmented design*: the hash table is internally composed of an array of submaps with internal locking (one mutex is created for each internal submap)
- *fine-grained locking*: separate locks for each internal submap provide a speed benefit when compared to single threaded insertion

The cache is then implemented as a `hash_map` object that maps Zobrist hash values to fitness. Integration with the workflow in Figure 1 is straightforward – the logic needs to be slightly adapted to only re-evaluate a solution if it has not been cached before, as seen in Algorithm 1. This is implemented in Operon at the level of the recombination operation. The cache is initially updated with the initial population created by the initialization step.

1: **procedure** CACHEDFITNESS(T)
2: $h \leftarrow \mathcal{H}(T)$
3: **if** `cache`$(h) = \emptyset$ **then**
4: `cache`$(h) \leftarrow$ EVALUATEFITNESS(T)
5: **end if**
6: **return** `cache`(h)
7: **end procedure**

Algorithm 1: Cached fitness evaluation

In addition to the hash map object, the following components are necessary:

Zobrist table The table is initialized with random values corresponding to each symbol × position combination, as illustrated in the right-hand side of figure Figure 5. Its shape will be an $n \times l$ matrix where the number of symbols n and the number of indices l are given by:

$$n = \underbrace{|\text{binary operators}| + |\text{unary operators}|}_{\text{function nodes}} + \underbrace{|\text{nullary operators}|}_{\text{terminal nodes}} \tag{3.1}$$

$$l = \text{the maximum tree length} \tag{3.2}$$

For example, if the primitive set contains 20 symbols and the maximum tree length is 50, then the Zobrist table will contain 1000 values. To avoid hash collisions, we use 64-bit values generated with a high-quality pseudo-random number generator[4] [43].

Zobrist hash function The hash function iterates over tree nodes in preorder and aggregates the corresponding values from the Zobrist table using the XOR operation:

$$\mathcal{H}(T) = \bigoplus_{i=1}^{|T|} h\left(s_i, i\right) \tag{3.3}$$

where T is a tree, s_i is the symbol at position i and $h(s_i, i)$ is a hash retrieval function:

$$h(s_i, i) = \begin{cases} \texttt{zobrist}(s_i, i) \oplus \texttt{id}(s_i) & \text{if } s_i \text{ is a variable} \\ \texttt{zobrist}(s_i, i) & \text{otherwise} \end{cases} \tag{3.4}$$

In the case of variables, each Zobrist table value is XOR'd with a unique identifier associated with each variable. This definition of h is necessary in order to treat variables as distinct symbols within this convention such that, for example, $\mathcal{H}(x_1) \neq \mathcal{H}\left(x_2\right)$.

Hashing of coefficients The Operon framework as well as many other SR implementations assign a multiplicative coefficient to each tree node in order to enable a more granular fine-tuning of the model using local search. Although these node coefficients can be included in the hash by XOR-ing their respective values into the hash, we believe this to be counter-productive as it would greatly decrease the efficiency of the cache. Therefore, we restrict the tree hash to nodes without coefficients, in effect only hashing tree structure, in order to lower memory requirements, prevent the cache from growing very large and avoid storing differently parameterized versions of the same tree structure.

4. Experimental Results

The purpose of our experiment is to ascertain the effectiveness of fitness caching in reducing runtime while maintaining solution quality. We therefore compare otherwise identical algorithmic configurations with and without fitness caching, using a number of real-world datasets illustrated in Table 1. We employ the Non-dominated Sorting Genetic Algorithm 2 (NSGA2) [44], a multi-objective evolutionary algorithm whose selection mechanism is based on the concepts of Pareto dominance and crowding distance, to allow the simultaneous optimization of model accuracy and size. The algorithm is configured with the hyperparameters described in Table 2. A single best model is selected from the returned Pareto front using the Minimum Description Length principle, which favors models that provide the shortest description of the data in an information-theoretical sense [45], therefore choosing the best compromise between error and complexity.

Since the individuals are cached based only on their structure without accounting for coefficients, we employ variable amounts of local search in order to ensure that model coefficients are properly optimized before fitness is evaluated. Recent research has shown that due to the

[4] We use the excellent general-purpose `xoshiro256**` generator from https://prng.di.unimi.it

Name	Features	Training size	Test size
Chemical-I [47]	25	3136	1863
Chemical-II [47]	57	711	355
Flow stress [48]	2	4200	3600
FrictionDyn [49]	17	1010	1016
FrictionStat [49]	16	1010	1016
Nasa Battery 1_10 [50]	6	504	132
Nasa Battery 2_20 [50]	5	1101	537
Nikuradse-I [51]	2	230	132
Nikuradse-II [51]	2	200	162

Table 1: Test problems

Fixed hyperparameters	**Value**
Population size	1000
Function set	$+, -, \times, \div, \exp, \text{logabs}, \sin, \text{sqrtabs}, \text{square}$
Terminal set	$\text{constants}, \text{variables}$
Tree initialization	balanced tree creator [41], max initial tree length = 10 nodes
Crossover probability	100%
Mutation probability	25%
Mutation operations	insert/remove/replace subtree, change function/variable/coefficient
Maximum generations	300
Maximum tree length	20
Maximum tree depth	10
Variable hyperparameters	**Value**
Local search iterations	$\{0, 1, 10, 50, 100\}$
Use transposition cache	{True, False}

Table 2: NSGA-2 Parameters

Baldwin effect, a relatively small amount of local search is sufficient in GP [31]. Operon performs a non-linear least squares optimization of model coefficients using the Levenberg-Marquardt algorithm [46], a Newton step-based gradient descent method where the step is adjusted to remain within a trust region using a damping factor λ. When the step is successful, the trust region is expanded by reducing λ. If the step is unsuccessful, the trust region is shrunk by increasing λ.

It is expected that a sufficient amount of local search will ensure that the fitness values stored in the cache are the best that can be attained for a given individual, countering the effect of hashing only the tree structure. The experiment includes a configuration with no local search (iterations set to zero), in order to analyze the effects of caching in the extreme case where no care is taken about the coefficients of the models. We perform 100 repetitions for each problem and set of hyperparameters, for a total of 9000 algorithmic runs.

Problem	Local Search Iterations				
	0	1	10	50	100
Chemical-I	0.817 ± 0.083	0.910 ± 0.009	0.920 ± 0.005	0.919 ± 0.005	0.920 ± 0.006
	0.835 ± 0.066	0.913 ± 0.006	0.922 ± 0.003	0.923 ± 0.003	0.922 ± 0.004
Chemical-II	0.553 ± 0.119	0.808 ± 0.082	0.802 ± 0.121	0.788 ± 0.164	0.794 ± 0.126
	0.569 ± 0.128	0.827 ± 0.040	0.805 ± 0.075	0.801 ± 0.129	0.807 ± 0.131
Flow-stress	0.951 ± 0.045	0.951 ± 0.043	0.999 ± 0.145	0.999 ± 0.071	0.999 ± 0.117
	0.944 ± 0.103	0.948 ± 0.048	0.999 ± 0.101	0.999 ± 0.087	0.999 ± 0.023
FrictionDyn	0.723 ± 0.077	0.856 ± 0.025	0.865 ± 0.007	0.863 ± 0.014	0.868 ± 0.007
	0.635 ± 0.120	0.869 ± 0.008	0.869 ± 0.006	0.872 ± 0.004	0.871 ± 0.005
FrictionStat	0.294 ± 0.186	0.843 ± 0.149	0.853 ± 0.046	0.863 ± 0.016	0.858 ± 0.027
	0.260 ± 0.222	0.843 ± 0.151	0.852 ± 0.068	0.863 ± 0.013	0.861 ± 0.022
Nasa Bat 1_10	0.983 ± 0.009	0.992 ± 0.004	0.993 ± 0.003	0.993 ± 0.004	0.993 ± 0.003
	0.986 ± 0.012	0.994 ± 0.002	0.994 ± 0.002	0.994 ± 0.002	0.994 ± 0.002
Nasa Bat 2_20	0.978 ± 0.068	0.972 ± 0.016	0.971 ± 0.005	0.972 ± 0.004	0.972 ± 0.005
	0.978 ± 0.053	0.973 ± 0.004	0.971 ± 0.005	0.971 ± 0.005	0.973 ± 0.005
Nikuradse-I	0.891 ± 0.131	0.921 ± 0.189	0.986 ± 0.208	0.989 ± 0.140	0.992 ± 0.125
	0.882 ± 0.134	0.944 ± 0.186	0.987 ± 0.133	0.993 ± 0.030	0.993 ± 0.058
Nikuradse-II	0.973 ± 0.124	0.982 ± 0.034	0.982 ± 0.020	0.982 ± 0.136	0.982 ± 0.044
	0.972 ± 0.156	0.982 ± 0.008	0.983 ± 0.000	0.982 ± 0.117	0.982 ± 0.136

Table 3: Test performance given as median test $R^2 \pm \sigma$, without caching (above) and with caching (below). Statistically significant differences, computed using a directional Mann-Whitney U test ($\alpha = 0.05$) are highlighted with green (better) and red (worse).

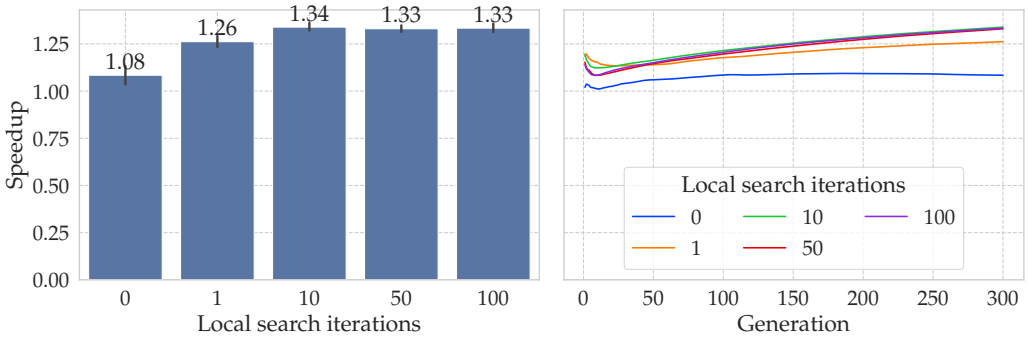
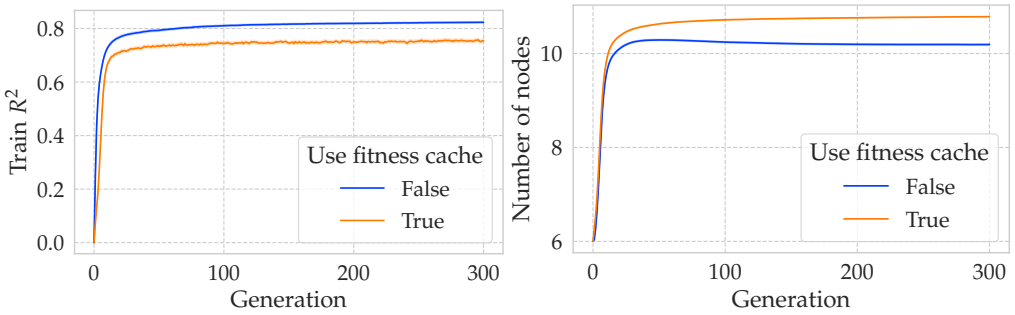


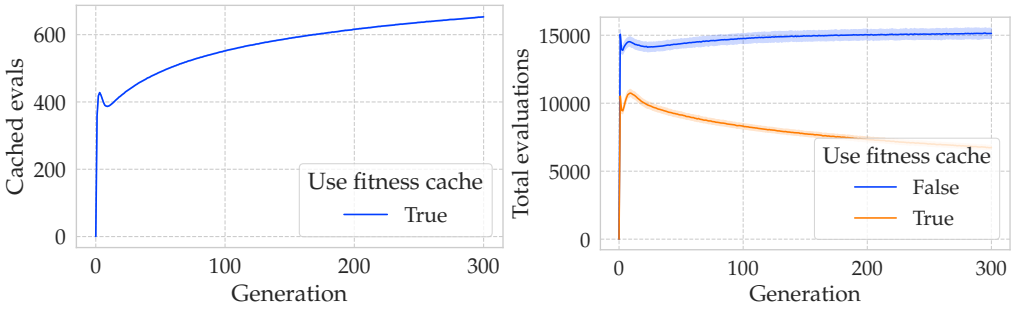
Figure 6: Average speedup obtained with caching at the end of run (left) and generational (right)

Results overview Experimental results demonstrate the runtime benefits of caching (illustrated in Figure 6) with observed speedups of up to 34%. At the same time, the median test R^2 values (illustrated in Table 3) show that caching does not have a detrimental effect on fitness, but rather tends to produce slightly better results. A Mann-Whitney U statistical test performed using the 100 R^2 test values obtained for each configuration confirms that, in a majority of cases, no statistically significant difference in the R^2 test values could be detected. The SR variant



(a) Average fitness (b) Average length

Figure 7: Average fitness and average length, with/without caching



(a) Cached evaluations per generation (b) Total evaluations per generation

Figure 8: Cached and total evaluations per generation

with caching achieves statistically significant better results (albeit with negligible differences in absolute terms) in 19 out of 45 test configurations (highlighted with green in Table 3), and statistically-worse results in 5 out of 45 test configurations (highlighted with red). It is worth mentioning that most statistically-worse results occur with an insufficient amount of local search (0 or 1 iterations). However, despite similar results, a more detailed analysis of the microevolutionary dynamics at play reveals a notable difference in search behavior with fitness caching. The full set of results is available online: https://github.com/foolnotion/symreg-london-2025.

Evolutionary dynamics Although the final models are just as good, SR with caching displays slightly higher best model size (overall average 20.46 nodes compared to 20.18 without caching). However, these results are significant only in 8 out of 45 configurations (3 lower, 5 higher best model length with caching). At the same time, Figure 7 clearly shows lower average fitness as well as higher average expression size with caching. A possible explanation for these differences is given by the fact that caching leads to a more uniform fitness landscape which in turn diminishes the effects of selection pressure. Since the NSGA2 algorithm selects for high fitness and low length, the results are lower average fitness and higher average length.

The effects of local search are significant even with a single iteration of gradient, further supporting the hypothesis of the Baldwin effect in GP.

Effects of caching We first measure "cache hits" (i.e. the number of times an individual's fitness is retrieved from the cache instead of being computed anew) and relate this quantity with the algorithm's population size of 1000 individuals. This indirectly represents a measure of population diversity, since a high cache hit rate implies duplicate individuals. The results shown in Figure 8a show that major diversity loss occurs in the first few generations of the search, followed by a more gradual increase of duplicate individuals. As the amount of duplicate individuals increases, more evaluation effort is saved due to caching, as illustrated in Figure 8b. We note that even with a population size of 1000 individuals, a much higher amount of evaluations is possible due to local search. Here, the total includes the computation of residuals and gradients for local search. Even though a large amount of re-evaluations is avoided due to caching, the observed speedup is markedly lower due to other algorithmic steps other than evaluation taking some part of the total runtime, as differences in search dynamics (for example, a higher average length with caching) and overheads of the caching mechanism.

5. Conclusion and Future Work

In this work we have shown that significant speedups of the evolutionary search in SR can be achieved via fitness caching. These improvements are made possible through the application of the Zobrist hash, a concept well-known in search algorithms for abstract board games. By indexing node positions and symbol types, the same concept is applicable for creating unique keys for the symbolic expressions that are encountered during the evolutionary search. However, its inability to detect semantic duplicates is an important limitation of this approach, that could be overcome in the future by introducing simplification heuristics for expressions, based on known identities and mathematical equivalences.

Our implementation based on the Operon framework utilizes a highly-optimized concurrent hash map library and is able to achieve up to 34% speedups (expected to increase with larger datasets) while maintaining solution quality. The high retrieval rate of fitness values from cache also suggests that diversity is rapidly lost in the initial phase of the evolutionary search. We also observe that fitness caching changes search behavior by altering the distribution of fitness values in the population, with notable effects in terms of average fitness and model length. This phenomenon is worth investigating further, as it could also lead to new mechanisms for directing the search and improving overall convergence.

Future research efforts will be focused in finding new applications for the Zobrist hash in other areas of the evolutionary algorithm, notably selection and recombination, such as: dynamic control of selection pressure, selecting for uniqueness alongside fitness and/or complexity, preventing duplicates during recombination or extending the caching concept to sub-expressions.

References

1. Rudin C, Chen C, Chen Z, Huang H, Semenova L, Zhong C. 2022 Interpretable machine learning: Fundamental principles and 10 grand challenges. *Statistics Surveys* **16**, 1 – 85. (10.1214/21-SS133)
2. Matchev KT, Matcheva K, Roman A. 2022 Analytical Modeling of Exoplanet Transit Spectroscopy with Dimensional Analysis and Symbolic Regression. *The Astrophysical Journal* **930**, 33. (10.3847/1538-4357/ac610c)
3. Bartlett, Deaglan J., Kammerer, Lukas, Kronberger, Gabriel, Desmond, Harry, Ferreira, Pedro G., Wandelt, Benjamin D., Burlacu, Bogdan, Alonso, David, Zennaro, Matteo. 2024 A precise symbolic emulator of the linear matter power spectrum. *A&A* **686**, A209. (10.1051/0004-6361/202348811)
4. Desmond H, Bartlett DJ, Ferreira PG. 2023 On the functional form of the radial acceleration relation. *Monthly Notices of the Royal Astronomical Society* **521**, 1817–1831. (10.1093/mnras/stad597)

5. Lemos P, Jeffrey N, Cranmer M, Ho S, Battaglia P. 2023 Rediscovering orbital mechanics with machine learning. *Machine Learning: Science and Technology* **4**, 045002. (10.1088/2632-2153/acfa63)
6. Grayeli A, Sehgal A, Costilla-Reyes O, Cranmer M, Chaudhuri S. 2024 Symbolic Regression with a Learned Concept Library. *ArXiv* **abs/2409.09359**.
7. Wang Y, Wagner N, Rondinelli JM. 2019 Symbolic regression in materials science. *MRS communications* **9**, 793–805. (10.1557/mrc.2019.85)
8. Wang C, Zhang Y, Wen C, Yang M, Lookman T, Su Y, Zhang TY. 2022 Symbolic regression in materials science via dimension-synchronous-computation. *Journal of Materials Science & Technology* **122**, 77–83. (10.1016/j.jmst.2021.12.052)
9. Wang G, Wang E, Li Z, Zhou J, Sun Z. 2024 Exploring the mathematic equations behind the materials science data using interpretable symbolic regression. *Interdisciplinary Materials* **3**, 637–657. (10.1002/idm2.12180)
10. Dong Z, Kong K, Matchev KT, Matcheva K. 2023 Is the machine smarter than the theorist: Deriving formulas for particle kinematics with symbolic regression. *Phys. Rev. D* **107**, 055018. (10.1103/PhysRevD.107.055018)
11. Morales-Alvarado M, Conde D, Bendavid J, Sanz V, Ubiali M. 2024 Symbolic regression for precision LHC physics. In *38th conference on Neural Information Processing Systems*.
12. Zhang Z, Chen Z. 2023 Modeling and Control of Robotic Manipulators Based on Symbolic Regression. *IEEE Transactions on Neural Networks and Learning Systems* **34**, 2440–2450. (10.1109/TNNLS.2021.3106648)
13. Danai K, La Cava WG. 2021 Controller design by symbolic regression. *Mechanical Systems and Signal Processing* **151**, 107348. (10.1016/j.ymssp.2020.107348)
14. La Cava WG, Lee PC, Ajmal I, Ding X, Solanki P, Cohen JB, Moore JH, Herman DS. 2023 A flexible symbolic regression method for constructing interpretable clinical prediction models. *npj Digital Medicine* **6**, Article number: 107. Silver 2023 HUMIES (10.1038/s41746-023-00833-8)
15. Petersen BK, Landajuela M, Mundhenk TN, Santiago CP, Kim S, Kim JT. 2021 Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.
16. Kamienny PA, d'Ascoli S, Lample G, Charton F. 2022 End-to-end symbolic regression with transformers. *Advances in Neural Information Processing Systems* **35**, 10269–10281.
17. Landajuela M, Lee CS, Yang J, Glatt R, Santiago CP, Aravena I, Mundhenk TN, Mulcahy G, Petersen BK. 2022 A Unified Framework for Deep Symbolic Regression. In Koyejo S, Mohamed S, Agarwal A, Belgrave D, Cho K, Oh A, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems, NeurIPS 2022* p. 33985–33998 New Orleans, USA. Curran Associates, Inc.
18. Vastl M, Kulhánek J, Kubalík J, Derner E, Babuška R. 2024 SymFormer: End-to-End Symbolic Regression Using Transformer-Based Architecture. *IEEE Access* **12**, 37840–37849. (10.1109/ACCESS.2024.3374649)
19. Sahoo S, Lampert C, Martius G. 2018 Learning equations for extrapolation and control. In *International Conference on Machine Learning* p. 4442–4450. Pmlr.
20. Worm T, Chiu K. 2013 Prioritized grammar enumeration: symbolic regression by dynamic programming. In Blum C, Alba E, Auger A, Bacardit J, Bongard J, Branke J, Bredeche N, Brockhoff D, Chicano F, Dorin A, Doursat R, Ekart A, Friedrich T, Giacobini M, Harman M, Iba H, Igel C, Jansen T, Kovacs T, Kowaliw T, Lopez-Ibanez M, Lozano JA, Luque G, McCall J, Moraglio A, Motsinger-Reif A, Neumann F, Ochoa G, Olague G, Ong YS, Palmer ME, Pappa GL, Parsopoulos KE, Schmickl T, Smith SL, Solnon C, Stuetzle T, Talbi EG, Tauritz D, Vanneschi L, editors, *GECCO '13: Proceeding of the fifteenth annual conference on Genetic and evolutionary computation conference* p. 1021–1028 Amsterdam, The Netherlands. ACM. Best paper (10.1145/2463372.2463486)
21. Cozad A, Sahinidis NV. 2018 A global MINLP approach to symbolic regression. *Mathematical Programming* **170**, 97–119. Special Issue: International Symposium on Mathematical Programming, Bordeaux, July 2018 (10.1007/s10107-018-1289-x)
22. McConaghy T. 2011 p. 235–260. In *FFX: Fast, Scalable, Deterministic Symbolic Regression Technology*, p. 235–260. New York, NY: Springer New York. (10.1007/978-1-4614-1770-5_13)
23. Kammerer L, Kronberger G, Burlacu B, Winkler SM, Kommenda M, Affenzeller M. 2020 p. 79–99. In *Symbolic Regression by Exhaustive Search: Reducing the Search Space Using Syntactical*

Constraints and Efficient Semantic Structure Deduplication, p. 79–99. Springer International Publishing. (10.1007/978-3-030-39958-0_5)
24. Koza JR. 1992 *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA, USA: MIT Press.
25. La Cava W, Orzechowski P, Burlacu B, de Franca F, Virgolin M, Jin Y, Kommenda M, Moore J. 2021 Contemporary Symbolic Regression Methods and their Relative Performance. In Vanschoren J, Yeung SK, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks* vol. 1. Curran.
26. de Franca FO, Virgolin M, Kommenda M, Majumder MS, Cranmer M, Espada G, Ingelse L, Fonseca A, Landajuela M, Petersen B, Glatt R, Mundhenk N, Lee CS, Hochhalter JD, Randall DL, Kamienny P, Zhang H, Dick G, Simon A, Burlacu B, Kasak J, Machado M, Wilstrup C, Cavaz WGL. 2024 SRBench++: Principled Benchmarking of Symbolic Regression With Domain-Expert Interpretation. *IEEE Transactions on Evolutionary Computation* p. 1–1. (10.1109/TEVC.2024.3423681)
27. Kronberger G, Olivetti de Franca F, Desmond H, Bartlett DJ, Kammerer L. 2024 The Inefficiency of Genetic Programming for Symbolic Regression. In Affenzeller M, Winkler SM, Kononova AV, Trautmann H, Tušar T, Machado P, Bäck T, editors, *Parallel Problem Solving from Nature – PPSN XVIII* p. 273–289 Cham. Springer Nature Switzerland.
28. Burks AR, Punch WF. 2015 An Efficient Structural Diversity Technique for Genetic Programming. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation* GECCO '15 p. 991–998 New York, NY, USA. Association for Computing Machinery. (10.1145/2739480.2754649)
29. Chen Q, Xue B, Zhang M. 2021 Preserving Population Diversity Based on Transformed Semantics in Genetic Programming for Symbolic Regression. *Trans. Evol. Comp* **25**, 433–447. (10.1109/TEVC.2020.3046569)
30. Kommenda M, Burlacu B, Kronberger G, Affenzeller M. 2020 Parameter identification for symbolic regression using nonlinear least squares. *Genetic Programming and Evolvable Machines* **21**, 471–501. Special Issue on Integrating numerical optimization methods with genetic programming (10.1007/s10710-019-09371-3)
31. Burlacu B, Winkler SM, Affenzeller M. 2024 Revisiting Gradient-Based Local Search in Symbolic Regression. In Winkler SM, Banzhaf W, Hu T, Lalejini A, editors, *Genetic Programming Theory and Practice XXI* Genetic and Evolutionary Computation p. 259–273 University of Michigan, USA. Springer. (10.1007/978-981-96-0077-9_13)
32. Črepinšek M, Liu SH, Mernik M. 2013 Exploration and exploitation in evolutionary algorithms: A survey. *ACM Comput. Surv.* **45**. (10.1145/2480741.2480752)
33. Prügel-Bennett A. 2000 Finite population effects for ranking and tournament selection. *Complex Systems* **12**, 183–205.
34. Xie H, Zhang M. 2011 Impacts of sampling strategies in tournament selection for genetic programming. *Soft Computing* **16**, 615–633.
35. Handley S. 1994 On the use of a directed acyclic graph to represent a population of computer programs. In *Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence* p. 154–159 vol.1. (10.1109/ICEC.1994.350024)
36. Keijzer M. 2004 Alternatives in Subtree Caching for Genetic Programming. In Keijzer M, O'Reilly UM, Lucas SM, Costa E, Soule T, editors, *Genetic Programming 7th European Conference, EuroGP 2004, Proceedings* vol. 3003*LNCS* p. 328–337 Coimbra, Portugal. Springer-Verlag. (10.1007/978-3-540-24650-3_31)
37. de Franca FO, Kronberger G. 2025 Improving Genetic Programming for Symbolic Regression with Equality Graphs. In *Proceedings of the Genetic and Evolutionary Computation Conference* GECCO '25 New York, NY, USA. Association for Computing Machinery. (10.1145/3712256.3726383)
38. Wong P, Zhang M. 2008 SCHEME: Caching subtrees in genetic programming. In *2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence)* p. 2678–2685. (10.1109/CEC.2008.4631158)
39. Zhang K, Statman R, Shasha D. 1992 On the editing distance between unordered labeled trees. *Information Processing Letters* **42**, 133–139. (10.1016/0020-0190(92)90136-J)
40. Burlacu B, Affenzeller M, Kronberger G, Kommenda M. 2019 Online Diversity Control in Symbolic Regression via a Fast Hash-based Tree Similarity Measure. In *2019 IEEE Congress on Evolutionary Computation (CEC)* p. 2175–2182. (10.1109/CEC.2019.8790162)

41. Burlacu B, Kronberger G, Kommenda M. 2020 Operon C++: an efficient genetic programming framework for symbolic regression. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion* GECCO '20 p. 1562–1570 New York, NY, USA. Association for Computing Machinery. (10.1145/3377929.3398099)
42. Popovitch G. 2019 The Parallel Hashmap. .
43. Blackman D, Vigna S. 2021 Scrambled Linear Pseudorandom Number Generators. *ACM Trans. Math. Softw.* **47**. (10.1145/3460772)
44. Deb K, Pratap A, Agarwal S, Meyarivan T. 2002 A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* **6**, 182–197. (10.1109/4235.996017)
45. Grünwald P, Roos T. 2019 Minimum description length revisited. *International Journal of Mathematics for Industry* **11**. (10.1142/s2661335219300018)
46. Gavin HP. 2019 The Levenberg-Marquardt algorithm for nonlinear least squares curve-fitting problems. *Department of Civil and Environmental Engineering Duke University August* **3**, 1–23.
47. Kordon A. 2008 Evolutionary Computation in the Chemical Industry. In Yu T, Davis D, Baydar C, Roy R, editors, *Evolutionary Computation in Practice*, Studies in Computational Intelligence, vol. 88, p. 245–262. Springer. (10.1007/978-3-540-75771-9_11)
48. Kabliman E, Kolody AH, Kronsteiner J, Kommenda M, Kronberger G. 2021 Application of symbolic regression for constitutive modeling of plastic deformation. *Applications in Engineering Science* **6**, 100052. (10.1016/j.apples.2021.100052)
49. Kronberger G, Kommenda M, Promberger A, Nickel F. 2018 Predicting friction system performance with symbolic regression and genetic programming with factor variables. In *Proceedings of the Genetic and Evolutionary Computation Conference* GECCO '18 p. 1278–1285 New York, NY, USA. Association for Computing Machinery. (10.1145/3205455.3205522)
50. Saha B, Goebel K. 2007 Battery data set. *NASA AMES prognostics data repository*.
51. Nikuradse J et al.. 1950 Laws of flow in rough pipes. .